Father Saturnino Urios University
College of Information Technology
Butuan City

WorkWise — Editorial Job Board Platform

A Django-based job board that connects employers and job seekers through role-based dashboards, a Kanban hiring pipeline, and admin moderation.

A Final Project Documentation
Presented to John Michael Tripoli
In Partial Fulfillment of the Requirements in
SIA 2 – Systems Integration and Architecture 2

Submitted by:
Group 2
Paredo, Celso
Acilo, Angel Mariel M.
Coral, Xean L.
Bugasia, Mil Kieven B.
Ramirez, Marrietta P.

Bachelor of Science in Information Technology, 4th yr & IT 43
1st Semester, AY 2026–2027
October 22, 2026

Table of Contents

Abstract....3
1. Introduction....4
1.1 Background of the Study....4
1.2 Statement of the Problem....4
1.3 Objectives of the Study....4
1.4 Scope and Limitations....5
1.5 Significance of the Study....5
2. System Overview....6
2.1 System Description....6
2.2 Users and Roles....6
2.3 Features and Modules....6
2.4 Technology Stack....7
3. System Architecture and Integration....8
3.1 System Architecture Diagram....8
3.2 Integration Approach....8
3.3 External APIs and Services Used....9
3.4 Data Flow....9
4. Database Design....11
4.1 Entity Relationship Diagram (ERD)....11
4.2 Data Dictionary....12
5. API Documentation....18
5.1 Base URL and Conventions....18
5.2 Endpoint Summary....18
5.3 Endpoint Details....19
6. Security Measures....22
7. User Interface and User Manual....23
7.1 Home Page....23
7.2 Login and Registration....23
7.3 Browse Jobs....24
7.4 Job Detail....25
7.5 Apply for a Job....26
7.6 My Applications....26
7.7 Applicant Dashboard and Profile....27
7.8 Employer Dashboard....28
7.9 Manage Jobs and Post a Job....29
7.10 Candidate Pipeline....29
7.11 Admin Panel....30
8. Testing....32
8.1 Testing Approach....32
8.2 Test Cases and Results....32
8.3 Known Issues and Fixes....33
9. Deployment and Installation Guide....35
9.1 System Requirements....35
9.2 Installation Steps....35
9.3 Source Code Repository....36
10. Team Roles and Contributions....37
11. Conclusion and Recommendations....38
11.1 Conclusion....38
11.2 Recommendations....38
References....39
Appendices....40

Abstract

Finding the right job, and the right candidate, remains slow when vacancies, applications and hiring decisions are scattered across emails, spreadsheets and chat threads. WorkWise is an editorial-style job board web application that gives applicants, employers and administrators a single platform for the whole hiring cycle. Built with Django 5, Vanilla CSS and HTMX on a SQLite database, the system provides role-based authentication, a searchable job board, employer job management with a live preview, an application flow with resume upload, and a Kanban candidate pipeline whose stage changes are enforced by a service-layer state machine. An admin panel supports user suspension, employer verification, job moderation and an audit log. Integration is achieved through Django's email framework, the EmailJS REST API for verification codes, and the HTMX library delivered from a CDN. Security measures include session authentication, rate-limited login, role and ownership checks, CSRF protection, validated resume uploads (PDF, DOC or DOCX up to 30 MB) and access-controlled media delivery. The system was verified with an automated suite of 128 test cases across five Django apps, all of which passed, and is packaged for deployment with Gunicorn, WhiteNoise and Docker. The result is a working end-to-end hiring platform that reduces manual tracking and gives every party clear visibility of application status.

Keywords: *job board, Django, HTMX, applicant tracking, role-based access control, systems integration*

1. Introduction

1.1 Background of the Study

Recruitment today involves many people and many channels. Employers advertise vacancies on notice boards, social media and email, then screen resumes by hand. Job seekers search several sites, send applications in different formats and rarely learn what happened to them afterward. Both sides lose time, and information such as resumes, interview schedules and decisions is easily misplaced.

WorkWise was developed as an integrated web platform for this problem. It serves three kinds of users: applicants who look for work, employers who post vacancies and review candidates, and administrators who keep the platform trustworthy. The system uses seeded demonstration data (for example the employers Synapse Labs and Orbit Systems and applicants such as Alex Rivera and Sam Chen) so that the complete hiring flow can be demonstrated without real personal data.

1.2 Statement of the Problem

The project addresses the following problems:

1. Applicants cannot easily discover vacancies filtered by category, location and salary in one place.
2. Applicants do not receive clear, timely feedback on the status of their applications or interview schedules.
3. Employers manage candidates in spreadsheets or email, which makes it hard to see who is at which stage of the hiring process.
4. Resumes and personal documents are shared through insecure channels, with no control over who may open them.
5. Platforms lack moderation tools to verify employers, remove inappropriate postings and keep a record of administrative actions.

1.3 Objectives of the Study

General objective: To design and develop WorkWise, a role-based job board web application that integrates job posting, application submission, candidate review and platform moderation in one secure system.

Specific objectives:

1. To implement authentication for three roles (Applicant, Employer, Admin) with login rate limiting (5 attempts per 5 minutes) and account suspension.
2. To provide employer job management with Draft, Open, Paused, Closed, Filled and Removed states, salary-range validation and a live preview rendered through HTMX.
3. To provide job discovery with category, location and salary filters and an application flow that accepts a cover letter and a resume (PDF, DOC or DOCX, maximum 30 MB) and allows only one application per job.
4. To implement a Kanban candidate pipeline in which every status change is checked by a single service-layer state machine and triggers a notification to the applicant.

5. To integrate an email verification-code service (EmailJS REST API with Django email fallback) for the password-reset flow.
6. To deliver an admin panel for user management, employer verification, job moderation and an audit log of administrative actions.
7. To verify the system with automated tests and prepare it for deployment with Gunicorn, WhiteNoise and Docker.

1.4 Scope and Limitations

Scope. The system covers user registration and login for three roles; role dashboards; public job browsing and search; employer job creation, editing, closing, reopening and removal; application submission, withdrawal and tracking; the per-job Kanban pipeline with candidate drawer, interview scheduling and outcome recording; in-app notifications; content reporting (flags); and the admin panel with user management, employer verification, job moderation and audit log.

Limitations. WorkWise is a web application only; there is no native mobile app. Demonstration data is created by a seed command. The default configuration uses SQLite; PostgreSQL support is mentioned in the project README but no PostgreSQL driver or settings are included in the repository. Outgoing email uses the console backend in development, and the SMTP backend must be configured for production. Features that load assets from external CDNs (HTMX, Google Fonts) need an internet connection. The system does not include payments, a public REST API or automated resume parsing.

1.5 Significance of the Study

- **Applicants** gain one place to search for jobs, apply with a resume, and follow their applications through a visual timeline and interview tickets.
- **Employers** gain a structured pipeline, candidate drawer and dashboard metrics that replace spreadsheets and scattered email.
- **Administrators** can verify employers, moderate jobs and review a full audit trail, which builds trust in the platform.
- **Future developers and students** receive a documented example of a layered Django project with a service layer, role mixins, HTMX partials, tests and a production-ready configuration.

2. System Overview

2.1 System Description

WorkWise is a server-rendered Django web application. A visitor can browse the home page and the job board without an account. After registering as an Applicant or an Employer (the role is chosen with a toggle on the sign-up page), the user is redirected to a role-specific dashboard.

A typical applicant journey is: sign up, complete the profile and upload a master resume, search jobs by category, location and salary, open a job and click Apply, write a cover letter and attach or reuse a resume, and then follow the application on the My Applications page as the employer moves it through the stages. Interview invitations appear as ticket cards on the dashboard and as notifications.

A typical employer journey is: register, wait for administrator verification (an unverified employer can save drafts but cannot publish jobs), post a job with a live preview, open the job's pipeline board, review candidates in the slide-over drawer, move cards between stages, schedule interviews and record outcomes. Administrators sign in to the admin panel at /admin-panel/ to verify employers, suspend or reactivate users, remove jobs and inspect the audit log.

2.2 Users and Roles

User Role	Description	What They Can Do
Applicant	A job seeker with an applicant profile (headline, skills, resume, links).	Browse and search jobs; apply once per job; withdraw an application; view the application timeline and interview tickets; edit the profile and master resume; receive notifications; report content.
Employer	A company representative with an employer profile. Status is Pending until verified by an admin.	Create, edit, close, reopen and remove own jobs (publish only when verified); view the per-job pipeline; move candidates between stages; schedule interviews and record results; download resumes of own applicants; view dashboard metrics.
Admin (Moderator / Superadmin)	Platform staff using the standalone admin panel. Access level is Moderator or Superadmin.	Review the verification queue and approve or reject employers; moderate and remove jobs; review flags; view the audit log; Superadmins can also suspend and reactivate users (Moderators can reactivate).

2.3 Features and Modules

Module / Feature	Description	User Role(s)
Authentication (accounts)	Registration with role toggle, email-or-username login, logout, role-based redirect, profile editing, change password, and a verification-code password-reset flow.	All
Role Dashboards (core)	Employer metrics (open jobs, applicants, shortlisted, interviews in 7 days); applicant status breakdown, profile completeness and upcoming interviews; admin overview.	Applicant, Employer, Admin
Job Management (jobs)	Job CRUD with status tabs, salary and deadline validation, categories, tags and a live HTMX preview card.	Employer

Module / Feature	Description	User Role(s)
Job Discovery (jobs)	Public job list with category, branch, location and salary filters, positions-by-field lookup and job detail pages.	Applicant, Visitor
Applications (applications)	Apply with cover letter and resume snapshot, duplicate prevention, withdraw, detail page and timeline.	Applicant
Candidate Pipeline (applications)	Kanban board, candidate drawer, state-machine transitions, interview scheduling, outcome logging and offers.	Employer
Notifications (core)	Persistent notifications with read state, mark-all-read and per-item read.	All
Moderation (moderation)	Content reports (flags), flag list and detail, resolution, and audit logging of admin actions.	Admin, All (report)
Admin Panel (core)	User management, employer verification queue, job moderation and audit log with filters.	Admin
Protected Media (core)	Access-controlled delivery of resumes: owner, hiring employer and admins only.	Applicant, Employer, Admin

2.4 Technology Stack

Layer	Technology / Tool	Version	Purpose
Frontend	Django templates, Vanilla CSS (inkboard.css), HTMX, Bootstrap 5 via crispy-bootstrap5	HTMX 1.9.12; crispy-bootstrap5 ≥ 2024.2	Server-rendered pages, partial updates (live preview, Kanban board, drawer), form rendering.
Backend	Python + Django	Python 3.11+ (3.12 in Docker); Django ≥ 5.0, < 6.0	Business logic, ORM, authentication, views, services.
Database	SQLite (default)	Bundled with Python	Stores users, jobs, applications and audit data. PostgreSQL is possible but not configured.
External APIs / Services	EmailJS REST API; Django SMTP/console email; unpkg CDN; Google Fonts	EmailJS API v1.0	Verification-code email, notification email, HTMX script, Sora/Inter/JetBrains Mono fonts.
Libraries	python-dotenv, WhiteNoise, Gunicorn, django-crispy-forms, Pillow, Faker	≥ 1.0, ≥ 6.6, ≥ 21.2, ≥ 2.1, ≥ 10.0, ≥ 24.0	Environment config, static files, WSGI server, forms, image fields, demo data.
Tools	Git/GitHub, Docker, Django test runner	—	Version control, containerisation, 128 automated tests.
Hosting / Deployment	localhost (development); Docker + Gunicorn (production)	python:3.12-slim image	Run locally with run server or deploy as a container with 3 Gunicorn workers.

3. System Architecture and Integration

3.1 System Architecture Diagram

WorkWise follows Django's model–template–view layering with an added service layer. The browser sends HTTP requests (and HTMX partial requests) that pass through Django's security middleware before reaching the role-protected views. Views delegate business rules such as pipeline transitions, moderation and notifications to the service layer, which uses the ORM to read and write the database. Resumes are not served directly: the /media/ route is handled by a protected view that checks ownership.

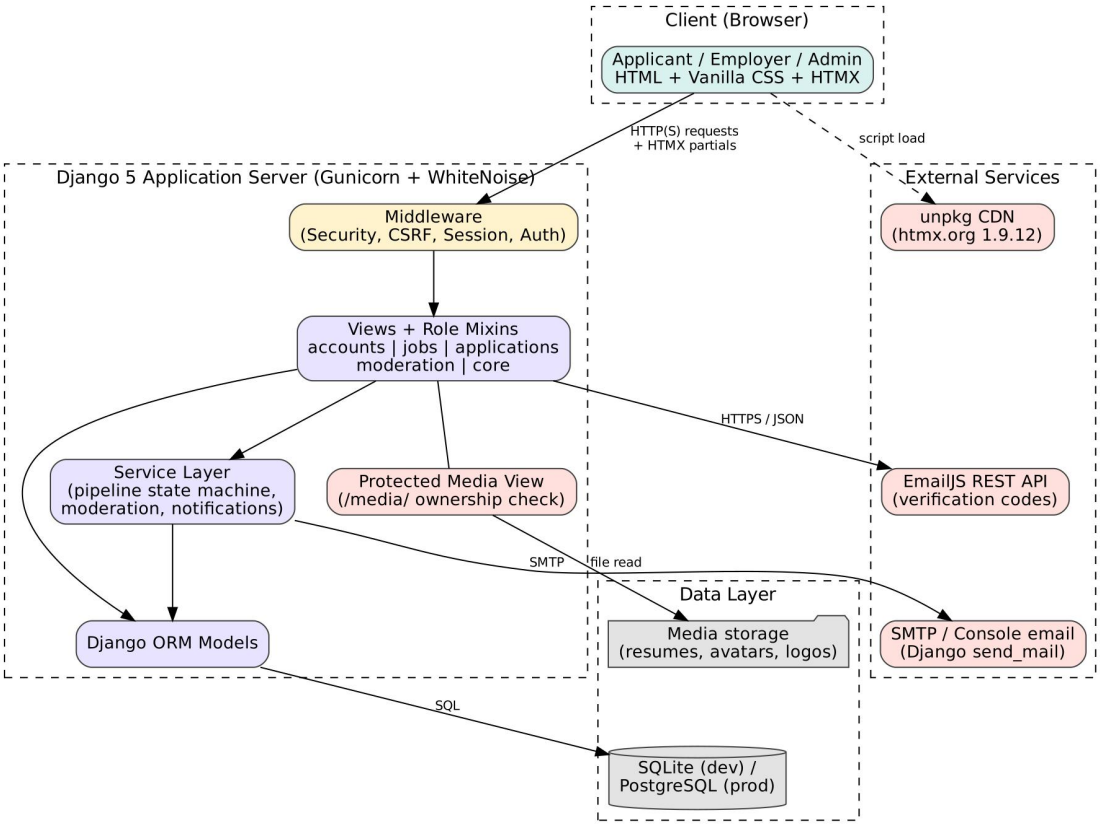


Figure 1. System Architecture Diagram

3.2 Integration Approach

The system uses a monolithic, layered architecture integrated with external services through simple, well-understood styles:

- **REST/JSON over HTTPS for EmailJS.** The password-reset flow generates a six-digit code and posts it to the EmailJS endpoint https://api.emailjs.com/api/v1.0/email/send. If the call fails, the system falls back to Django's send_mail and can return a payload for client-side delivery.
- **SMTP through Django's email framework** for pipeline notifications such as status changes. The console backend is used in development and the SMTP backend in production.
- **HTML-over-the-wire with HTMX.** Instead of a separate JSON API and single-page app, interactions such as the job preview, Kanban board refresh, drawer and

notifications request HTML fragments from Django views. This keeps one codebase, reuses server-side validation and suits a small team.

- **Shared database via the ORM.** The five Django apps (accounts, jobs, applications, moderation, core) integrate through foreign keys and a generic relation used by flags and audit logs.

3.3 External APIs and Services Used

API / Service	Provider	Purpose in the System	Auth Method
EmailJS REST API	EmailJS	Sends the six-digit verification code used for password reset.	Service/template IDs with public and private keys
SMTP / console email	Django (django.core.mail)	Sends status-change and notification emails.	Backend settings (EMAIL_BACKEND)
htmx.org 1.9.12	unpkg CDN	Provides HTMX for live preview, Kanban and drawer interactions.	None (public script)
Google Fonts	Google	Loads Sora, Inter and JetBrains Mono typefaces.	None (public stylesheet)

3.4 Data Flow

Figure 2 shows the data flow for the key process of the system: an applicant applies for a job and an employer moves the application through the pipeline.

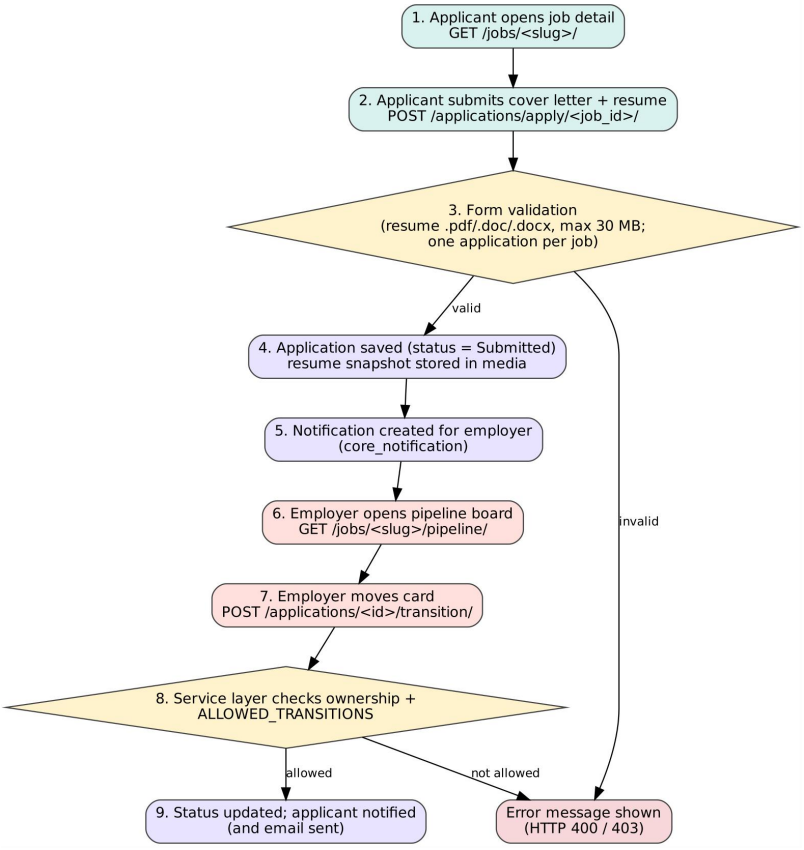


Figure 2. Data Flow Diagram: Apply and Review Process

The applicant opens a job (GET /jobs/<slug>/) and submits the form (POST /applications/apply/<job_id>/). The form validates the resume type and size and the system

rejects duplicate applications. A valid application is stored with status Submitted and a snapshot of the resume, and a notification is created for the employer. The employer opens the pipeline board and moves a card (POST /applications/<id>/transition/). The service layer confirms that the user owns the job and that the move is listed in ALLOWED_TRANSITIONS; if so the status is updated and the applicant is notified by an in-app notification and email, otherwise an error response is returned.

4. Database Design

4.1 Entity Relationship Diagram (ERD)

The database has twelve application tables. Figure 3 shows the core hiring entities; Figure 4 shows the supporting tables that reference the user. All relationships are enforced with foreign keys. An applicant can apply to a given job only once (unique pair applicant_id + job_id), and an application can have at most one offer.

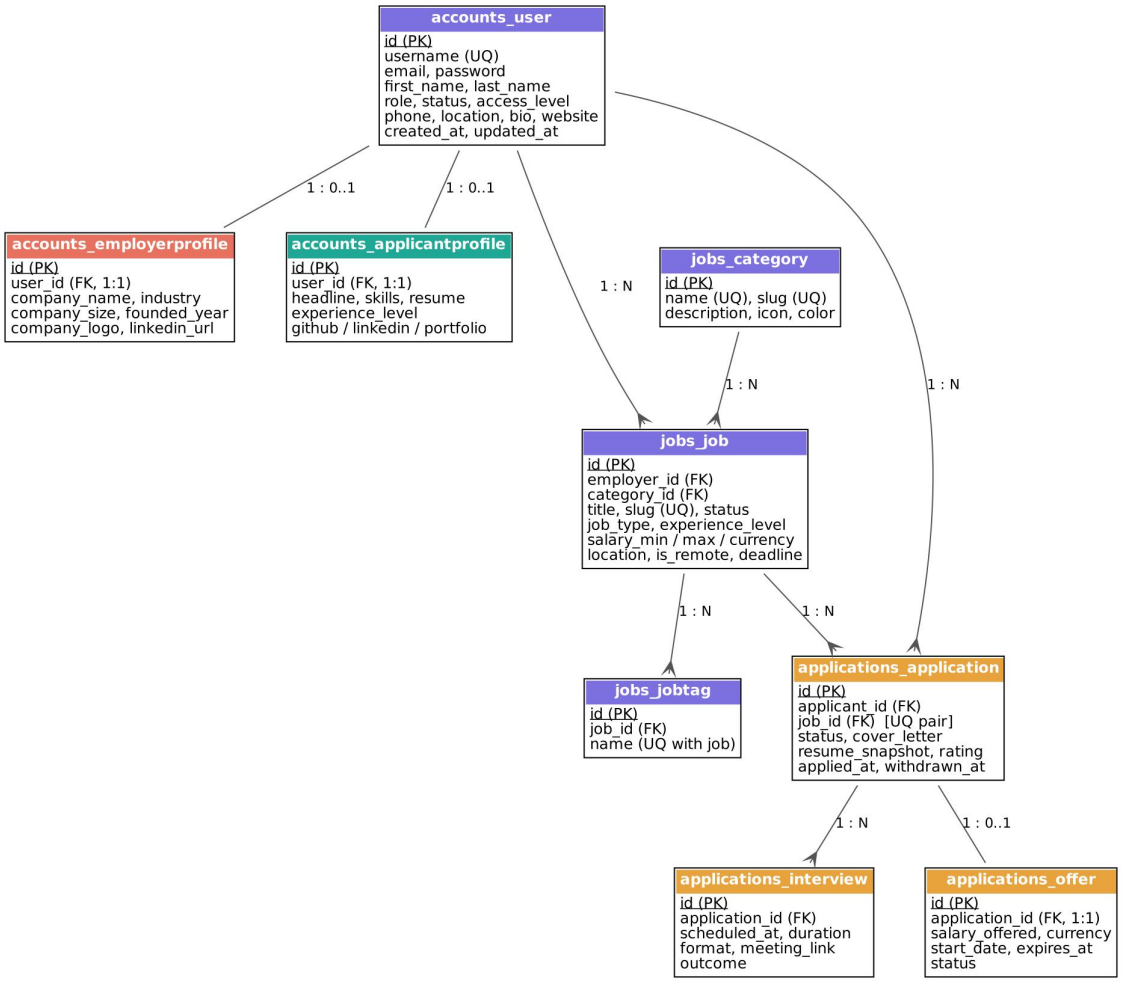


Figure 3. Entity Relationship Diagram: Core Tables

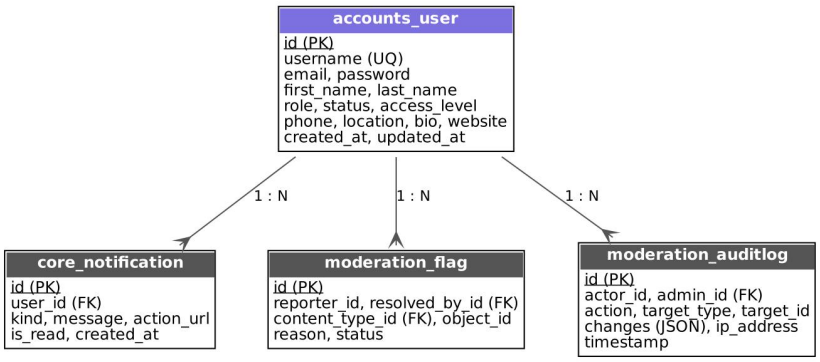


Figure 4. Entity Relationship Diagram: Notification and Moderation Tables

4.2 Data Dictionary

The tables below list every field generated from the Django models. Django's built-in tables (sessions, content types, permissions and groups) are omitted. Sizes are shown for variable-length fields; DECIMAL sizes are precision, scale.

Table name: accounts_user

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
password	VARCHAR	128	NOT NULL	Hashed password (PBKDF2).
last_login	DATETIME	—	NULL	Date/time of last login.
is_superuser	BOOLEAN	—	NOT NULL	Django superuser flag.
username	VARCHAR	150	UNIQUE, NOT NULL	Unique login name.
first_name	VARCHAR	150	NOT NULL	Given name.
last_name	VARCHAR	150	NOT NULL	Surname.
email	VARCHAR	254	NOT NULL	Email address (also usable for login).
is_staff	BOOLEAN	—	NOT NULL	Django staff flag.
is_active	BOOLEAN	—	NOT NULL	Whether the account is active.
date_joined	DATETIME	—	NOT NULL	Date/time the account was created.
role	VARCHAR	20	NOT NULL	User role: admin, employer or applicant.
access_level	VARCHAR	20	NOT NULL	Admin level: moderator or superadmin.
status	VARCHAR	20	NOT NULL	Account or record status.
avatar	VARCHAR	100	NULL	Profile picture path.
bio	TEXT	—	NOT NULL	Short biography.
phone	VARCHAR	30	NOT NULL	Contact number.
location	VARCHAR	120	NOT NULL	Location text.
website	VARCHAR	200	NOT NULL	Personal or company website.
created_at	DATETIME	—	NOT NULL	Date/time created.
updated_at	DATETIME	—	NOT NULL	Date/time last updated.

Table name: accounts_employerprofile

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
user_id	BIGINT	—	FK → accounts_user, UNIQUE, NOT NULL	Owning user (FK).
company_name	VARCHAR	200	NOT NULL	Employer company name.
company_logo	VARCHAR	100	NULL	Company logo path.
company_description	TEXT	—	NOT NULL	About the company.
company_size	VARCHAR	20	NOT NULL	Company size bracket.
industry	VARCHAR	120	NOT NULL	Industry or sector.

Field Name	Data Type	Size	Key / Constraint	Description
founded_year	SMALLINT	—	NULL	Year the company was founded.
linkedin_url	VARCHAR	200	NOT NULL	LinkedIn profile link.

Table name: accounts_applicantprofile

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
user_id	BIGINT	—	FK → accounts_user, UNIQUE, NOT NULL	Owning user (FK).
headline	VARCHAR	200	NOT NULL	Professional headline.
resume	VARCHAR	100	NULL	Master resume file path.
experience_level	VARCHAR	20	NOT NULL	Experience level.
skills	TEXT	—	NOT NULL	Comma-separated skills.
available_from	DATE	—	NULL	Date the applicant is available to start.
linkedin_url	VARCHAR	200	NOT NULL	LinkedIn profile link.
github_url	VARCHAR	200	NOT NULL	GitHub profile link.
portfolio_url	VARCHAR	200	NOT NULL	Portfolio link.

Table name: jobs_category

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
name	VARCHAR	100	UNIQUE, NOT NULL	Name / label.
slug	VARCHAR	120	UNIQUE, NOT NULL	URL-safe unique slug.
description	TEXT	—	NOT NULL	Description text.
icon	VARCHAR	60	NOT NULL	Icon name.
color	VARCHAR	7	NOT NULL	Hex colour code.
created_at	DATETIME	—	NOT NULL	Date/time created.

Table name: jobs_job

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
employer_id	BIGINT	—	FK → accounts_user, NOT NULL	Employer who posted the job (FK).
category_id	BIGINT	—	FK → jobs_category, NULL	Job category (FK).
title	VARCHAR	200	NOT NULL	Job title.
slug	VARCHAR	220	UNIQUE, NOT NULL	URL-safe unique slug.
description	TEXT	—	NOT NULL	Description text.
requirements	TEXT	—	NOT NULL	Job requirements.

Field Name	Data Type	Size	Key / Constraint	Description
responsibilities	TEXT	—	NOT NULL	Job responsibilities.
benefits	TEXT	—	NOT NULL	Benefits offered.
status	VARCHAR	20	NOT NULL	Account or record status.
job_field	VARCHAR	100	NOT NULL	Job field / specialisation.
uniformed_branch	VARCHAR	20	NOT NULL	Branch for uniformed-personnel jobs.
department	VARCHAR	120	NOT NULL	Department.
job_type	VARCHAR	20	NOT NULL	Employment type.
experience_level	VARCHAR	20	NOT NULL	Experience level.
location	VARCHAR	200	NOT NULL	Location text.
is_remote	BOOLEAN	—	NOT NULL	Whether the job is remote.
salary_min	DECIMAL	12,2	NULL	Minimum salary.
salary_max	DECIMAL	12,2	NULL	Maximum salary.
salary_period	VARCHAR	20	NOT NULL	Salary period (e.g. monthly).
salary_currency	VARCHAR	5	NOT NULL	Salary currency code.
deadline	DATE	—	NULL	Application deadline.
created_at	DATETIME	—	NOT NULL	Date/time created.
updated_at	DATETIME	—	NOT NULL	Date/time last updated.
views_count	INTEGER	—	NOT NULL	Number of listing views.

Table name: jobs_jobtag

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
job_id	BIGINT	—	FK → jobs_job, NOT NULL	Related job (FK).
name	VARCHAR	60	NOT NULL	Name / label.

Table name: applications_application

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
applicant_id	BIGINT	—	FK → accounts_user, NOT NULL	Applicant user (FK).
job_id	BIGINT	—	FK → jobs_job, NOT NULL	Related job (FK).
cover_letter	TEXT	—	NOT NULL	Cover letter text.
resume_snapshot	VARCHAR	100	NULL	Resume file captured at application time.
portfolio_url	VARCHAR	200	NOT NULL	Portfolio link.
status	VARCHAR	30	NOT NULL	Account or record status.
employer_notes	TEXT	—	NOT NULL	Private employer notes.

Field Name	Data Type	Size	Key / Constraint	Description
rating	SMALLINT	—	NULL	Employer rating of the candidate.
applied_at	DATETIME	—	NOT NULL	Date/time applied.
updated_at	DATETIME	—	NOT NULL	Date/time last updated.
withdrawn_at	DATETIME	—	NULL	Date/time withdrawn.

Table name: applications_interview

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
application_id	BIGINT	—	FK → applications_application, NOT NULL	Related application (FK).
scheduled_at	DATETIME	—	NOT NULL	Interview date/time.
duration_minutes	SMALLINT	—	NOT NULL	Interview length in minutes.
format	VARCHAR	20	NOT NULL	Interview format (video, phone, on-site).
meeting_link	VARCHAR	200	NOT NULL	Meeting URL.
location	VARCHAR	200	NOT NULL	Location text.
interviewer_notes	TEXT	—	NOT NULL	Interviewer notes.
outcome	VARCHAR	20	NOT NULL	Interview outcome.
created_at	DATETIME	—	NOT NULL	Date/time created.

Table name: applications_offer

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
application_id	BIGINT	—	FK → applications_application, UNIQUE, NOT NULL	Related application (FK).
salary_offered	DECIMAL	12,2	NOT NULL	Offered salary.
currency	VARCHAR	5	NOT NULL	Currency code.
start_date	DATE	—	NULL	Proposed start date.
expires_at	DATETIME	—	NULL	Offer expiry.
offer_letter	VARCHAR	100	NULL	Offer letter file path.
status	VARCHAR	20	NOT NULL	Account or record status.
notes	TEXT	—	NOT NULL	Notes.
created_at	DATETIME	—	NOT NULL	Date/time created.
updated_at	DATETIME	—	NOT NULL	Date/time last updated.

Table name: core_notification

Field Name	Data Type	Size	Key / Constraint	Description

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
user_id	BIGINT	—	FK → accounts_user, NOT NULL	Owning user (FK).
kind	VARCHAR	20	NOT NULL	Notification type.
message	VARCHAR	255	NOT NULL	Notification message.
highlight	VARCHAR	120	NOT NULL	Highlighted text.
detail_title	VARCHAR	255	NOT NULL	Detail panel title.
description	TEXT	—	NOT NULL	Description text.
status_label	VARCHAR	60	NOT NULL	Status label.
pills	JSON	—	NOT NULL	Tag pills (JSON).
action_url	VARCHAR	255	NOT NULL	Link opened by the notification.
action_text	VARCHAR	60	NOT NULL	Link label.
is_read	BOOLEAN	—	NOT NULL	Read flag.
read_at	DATETIME	—	NULL	Date/time read.
created_at	DATETIME	—	NOT NULL	Date/time created.

Table name: moderation_flag

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
reporter_id	BIGINT	—	FK → accounts_user, NULL	User who reported the content (FK).
content_type_id	BIGINT	—	FK → django_content_type, NOT NULL	Content type of the target (FK).
object_id	BIGINT	—	NOT NULL	Primary key of the reported/target object.
reason	VARCHAR	30	NOT NULL	Reason for the report.
description	TEXT	—	NOT NULL	Description text.
status	VARCHAR	20	NOT NULL	Account or record status.
resolved_by_id	BIGINT	—	FK → accounts_user, NULL	Admin who resolved the flag (FK).
resolution_notes	TEXT	—	NOT NULL	Resolution notes.
created_at	DATETIME	—	NOT NULL	Date/time created.
resolved_at	DATETIME	—	NULL	Date/time resolved.

Table name: moderation_auditlog

Field Name	Data Type	Size	Key / Constraint	Description
id	BIGINT	—	PK, AUTO_INCREMENT, NOT NULL	Unique identifier (auto-increment).
actor_id	BIGINT	—	FK → accounts_user, NULL	User who performed the action (FK).

Field Name	Data Type	Size	Key / Constraint	Description
admin_id	BIGINT	—	FK → accounts_user, NULL	Admin who performed the action (FK).
content_type_id	BIGINT	—	FK → django_content_type, NULL	Content type of the target (FK).
object_id	BIGINT	—	NULL	Primary key of the reported/target object.
target_type	VARCHAR	50	NOT NULL	Type of the affected record.
target_id	BIGINT	—	NULL	ID of the affected record.
object_repr	VARCHAR	400	NOT NULL	Readable representation of the object.
action	VARCHAR	30	NOT NULL	Action performed.
changes	JSON	—	NOT NULL	JSON of changed values.
ip_address	VARCHAR	39	NULL	IP address of the actor.
timestamp	DATETIME	—	NOT NULL	Date/time of the action.

5. API Documentation

WorkWise is a server-rendered application and does not expose a separate JSON REST API. Its public interface is the set of HTTP routes below, which return HTML pages, HTML fragments (for HTMX requests) or redirects. The only JSON-style endpoint is the notification state. The system also consumes one external API (EmailJS), described in Section 3.3.

5.1 Base URL and Conventions

Base URL: http://localhost:8000 (development). Route groups: /accounts/, /jobs/, /applications/, /moderation/, /media/ and the root routes of the core app.

- **Data format:** HTML responses; form-encoded POST bodies (multipart for file uploads). HTMX requests send the HX-Request header and receive HTML fragments.
- **Authentication:** Django session cookie. State-changing requests require the CSRF token (csrfmiddlewaretoken form field or X-CSRFToken header).
- **Errors:** 400 for invalid input or an invalid transition; 403 (custom page) when the role or ownership check fails; 404 (custom page) for unknown objects; 405 for the wrong method; 429 after too many login attempts.

5.2 Endpoint Summary

Method	Endpoint	Description	Auth Required
GET	/	Home page	No
GET	/jobs/	Job list with filters (category, branch, location, salary); HTMX partial results	No
GET	/jobs/<slug>/	Job detail page	No
GET/POST	/accounts/login/	Login by email or username (rate-limited)	No
GET/POST	/accounts/register/	Register as Applicant or Employer	No
POST	/accounts/logout/	Log out	Yes
GET/POST	/accounts/profile/	View and edit own profile	Yes
GET/POST	/accounts/password-reset/	Start password reset and send verification code	No
POST	/accounts/password-reset/verify/	Verify the six-digit code	No
GET/POST	/accounts/change-password/	Change password	Yes
GET/POST	/jobs/post/	Create a job	Yes (Employer)
GET	/jobs/manage/	List own jobs by status tab	Yes (Employer)
POST	/jobs/preview/	HTMX live preview card of a job	Yes (Employer)
GET/POST	/jobs/<slug>/edit/	Edit a job	Yes (Owner)
POST	/jobs/<slug>/close/ , /reopen/ , /delete/	Close, reopen or soft-delete a job	Yes (Owner)
GET	/jobs/<slug>/pipeline/	Kanban pipeline board for a job	Yes (Owner)
GET/POST	/applications/apply/<job_id>/	Submit an application	Yes (Applicant)

Method	Endpoint	Description	Auth Required
GET	/applications/	My applications list	Yes (Applicant)
GET	/applications/<id>/	Application detail and timeline	Yes
POST	/applications/<id>/withdraw/	Withdraw an application	Yes (Applicant)
POST	/applications/<id>/transition/	Move application to a new status	Yes (Owner)
GET	/applications/<id>/drawer/	Candidate drawer fragment	Yes (Owner)
POST	/applications/<id>/schedule-interview/	Schedule an interview	Yes (Owner)
POST	/applications/interviews/<id>/record-result/	Record interview outcome	Yes (Owner)
GET	/applications/<id>/resume/	Download resume of an application	Yes (Owner/Applicant/Admin)
GET	/media/<path>	Protected media file	Yes (restricted)
GET	/notifications/state/	Notification state (JSON)	Yes
POST	/notifications/mark-all-read/ , /notifications/<id>/read/	Mark notifications as read	Yes
GET	/applicant/ , /employer/ , /dashboard/	Role dashboards	Yes
GET	/admin-panel/ (users, verifications, jobs, audit-log)	Admin panel pages	Yes (Admin)
POST	/admin-panel/users/<id>/suspend/ , /reactivate/	Suspend or reactivate a user	Yes (Admin)
POST	/admin-panel/verifications/<id>/approve/ , /reject/	Approve or reject an employer	Yes (Admin)
POST	/admin-panel/jobs/<id>/remove/	Remove a job (moderation)	Yes (Admin)
GET/POST	/moderation/report/<type>/<id>/	Report content	Yes
GET	/moderation/flags/ , /moderation/flags/<id>/	Flag list and detail	Yes (Admin)

5.3 Endpoint Details

Endpoint: POST /accounts/login/

Request parameters / body

```
username   (email address or username)
password
csrfmiddlewaretoken
```

Sample request

```
POST /accounts/login/ HTTP/1.1
Content-Type: application/x-www-form-urlencoded

username=applicant1@workwise.dev&password=applicant1234&csrfmiddlewaretoken=...
```

Sample response

```
HTTP/1.1 302 Found
Location: /accounts/redirect/   (then to /applicant/, /employer/ or /admin-panel/)
Set-Cookie: sessionid=...; HttpOnly
```

Status codes

```
302 login succeeded | 200 form re-rendered with error | 429 more than 5 attempts in 5
minutes
```

Endpoint: POST /applications/apply/<job_id>/

Request parameters / body

```
cover_letter  (text)
resume        (file: .pdf .doc .docx, max 30 MB; optional if profile resume is used)
portfolio_url (URL, optional)
```

Sample request

```
POST /applications/apply/114/ HTTP/1.1
Content-Type: multipart/form-data; boundary=----x

cover_letter=I am excited to apply...  resume=@cv.pdf
```

Sample response

```
HTTP/1.1 302 Found
Location: /applications/<new_id>/
(flash message: "Application for '<job title>' submitted successfully!")
```

Status codes

```
302 created | 200 already-applied page or validation errors | 400 job not open or
deadline passed | 403 not an applicant | 404 unknown job
```

Endpoint: POST /applications/<id>/transition/

Request parameters / body

```
new_status  one of: under_review, shortlisted, interview_scheduled,
            offer_extended, hired, rejected
```

Sample request

```
POST /applications/12/transition/ HTTP/1.1
HX-Request: true
Content-Type: application/x-www-form-urlencoded

new_status=shortlisted&csrfmiddlewaretoken=...
```

Sample response

```
HTTP/1.1 200 OK   (HTMX request: re-rendered pipeline board HTML fragment)
HTTP/1.1 302 Found  (normal request: redirect to /jobs/<slug>/pipeline/)
```

Status codes

```
200/302 moved | 400 missing new_status or invalid transition | 403 not the job's
employer | 405 not POST
```

Endpoint: POST /applications/<id>/schedule-interview/

Request parameters / body

```
scheduled_at (date-time, required)
mode (video | phone | onsite), duration_minutes (default 45)
meeting_link, location, notes (optional)
```

Sample request

```
POST /applications/12/schedule-interview/
```

```
scheduled_at=2026-11-
05T10:00&mode=video&duration_minutes=45&meeting_link=https://meet.example/abc
```

Sample response

```
HTTP/1.1 302 Found  Location: /jobs/<slug>/pipeline/   (or HTMX board fragment)
Application status becomes interview_scheduled; applicant is notified.
```

Status codes

```
200/302 scheduled | 400 missing or invalid date | 403 not the job's employer | 405 not
POST
```

Endpoint: GET /media/<path>

Request parameters / body

```
path   relative path of the stored file, for example resumes/cv.pdf
```

Sample request

```
GET /media/resumes/cv.pdf HTTP/1.1   (with session cookie)
```

Sample response

```
HTTP/1.1 200 OK  (file bytes)  when the user is the owning applicant,
the hiring employer, or an administrator
```

Status codes

```
200 file served | 403 user is not allowed to view this document | 404 file not found
```

6. Security Measures

WorkWise applies defence in depth: Django's built-in protections are combined with role checks, rate limiting and controlled file access.

Security Concern	How Our System Handles It
Authentication	Django session authentication with a custom EmailBackend (login by email or username). Login attempts are limited to 5 per 5 minutes per IP address and then return HTTP 429. Suspended accounts are blocked. Password reset requires a six-digit verification code with an attempt counter stored in the session.
Authorization / Roles	Role mixins (EmployerRequiredMixin, ApplicantRequiredMixin, AdminRequiredMixin) plus a suspension check protect views. Object-level checks ensure only the owning employer can edit a job or move its candidates. Admin actions depend on access level (only Superadmins suspend users). Unauthorized access returns a custom 403 page.
Password Storage	Passwords are hashed by Django (PBKDF2) and checked by four validators: similarity, minimum length, common password and numeric-only.
Input Validation	Django forms and model validation: salary minimum must not exceed maximum, deadlines cannot be in the past, and resumes must be .pdf, .doc or .docx up to 30 MB. The ORM uses parameterised queries (SQL injection protection), templates auto-escape output (XSS protection) and CsrfViewMiddleware protects all POST requests. Application state changes are limited to the ALLOWED_TRANSITIONS map.
API Keys / Secrets	SECRET_KEY, DEBUG and ALLOWED_HOSTS are read from a .env file by python-dotenv. EmailJS identifiers are defined in accounts/email_service.py; see Section 8.3 and 11.2 for the recommendation to move them to environment variables before public release.
Data in Transit	Production settings enable secure and HttpOnly session and CSRF cookies, HSTS (one year, include subdomains, preload), X-Content-Type-Options nosniff and X-Frame-Options DENY. TLS is expected to be terminated by the hosting reverse proxy. External calls (EmailJS, CDNs) use HTTPS.
File Access	Uploaded resumes are served only through protected_media_serve, which allows the owning applicant, the hiring employer and administrators, and denies everyone else.
Auditability	Administrative actions (suspension, verification, job removal) are recorded in the audit log with actor, target, changes and IP address.

7. User Interface and User Manual

This section walks through the main screens using screenshots captured from the running system with the seeded demonstration data.

7.1 Home Page

The landing page introduces WorkWise and links to job search, sign-up and login.

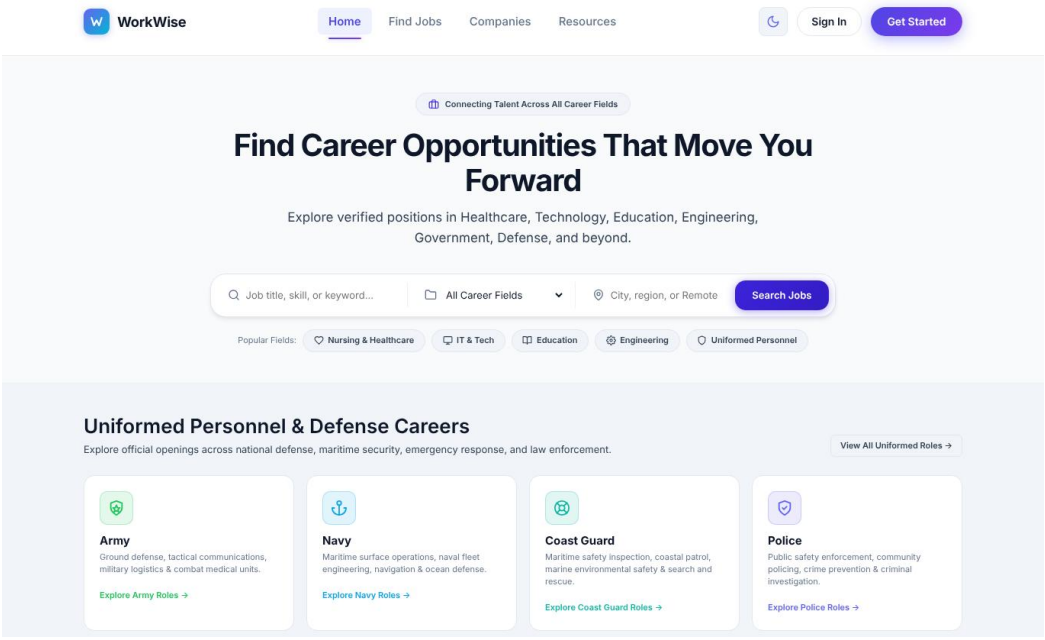


Figure 5. Home Page

Steps:

1. Open the site root (http://localhost:8000/).
2. Use the navigation bar to open Find Jobs, Companies or Resources.
3. Select Sign in or Sign up to access your account.

7.2 Login and Registration

Split-screen authentication pages. The role tabs change colour (teal for Applicants, coral for Employers).

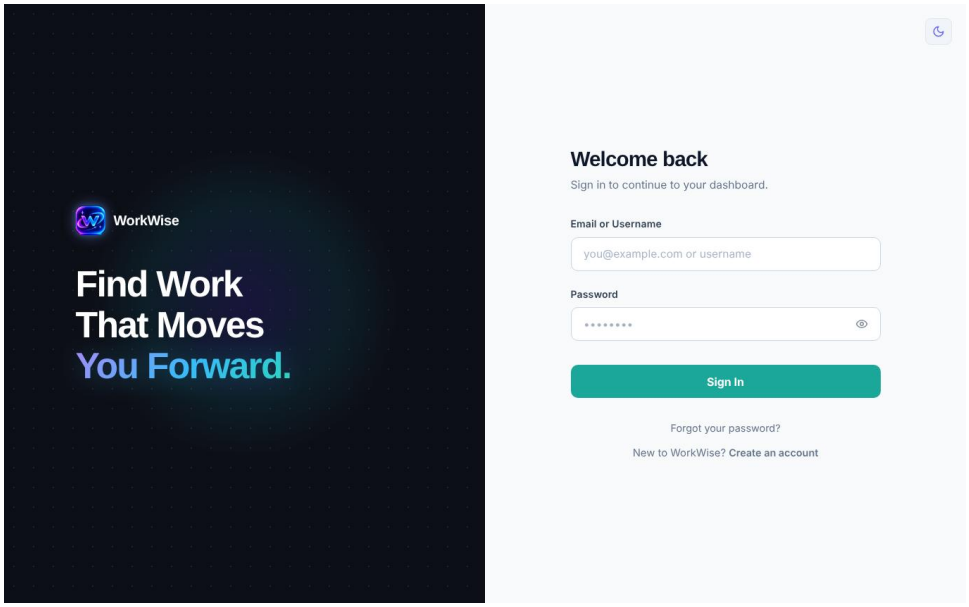


Figure 6. Login Page

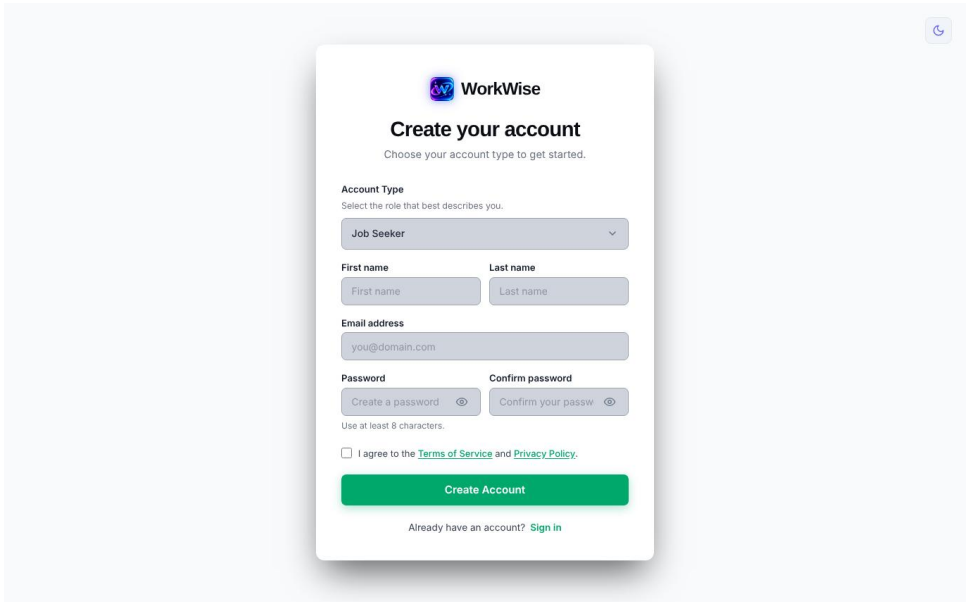


Figure 7. Registration Page with Role Switch

Steps:

1. Open Sign in and enter your email (or username) and password, then submit. After five failed attempts within five minutes the page asks you to wait.
2. To register, choose Applicant or Employer with the role toggle, fill in your details and submit.
3. Employers see a pending-verification banner until an administrator approves the account.
4. Use Forgot password to receive a six-digit code by email and set a new password.

7.3 Browse Jobs

The public job board lists open vacancies with filters and instant search.

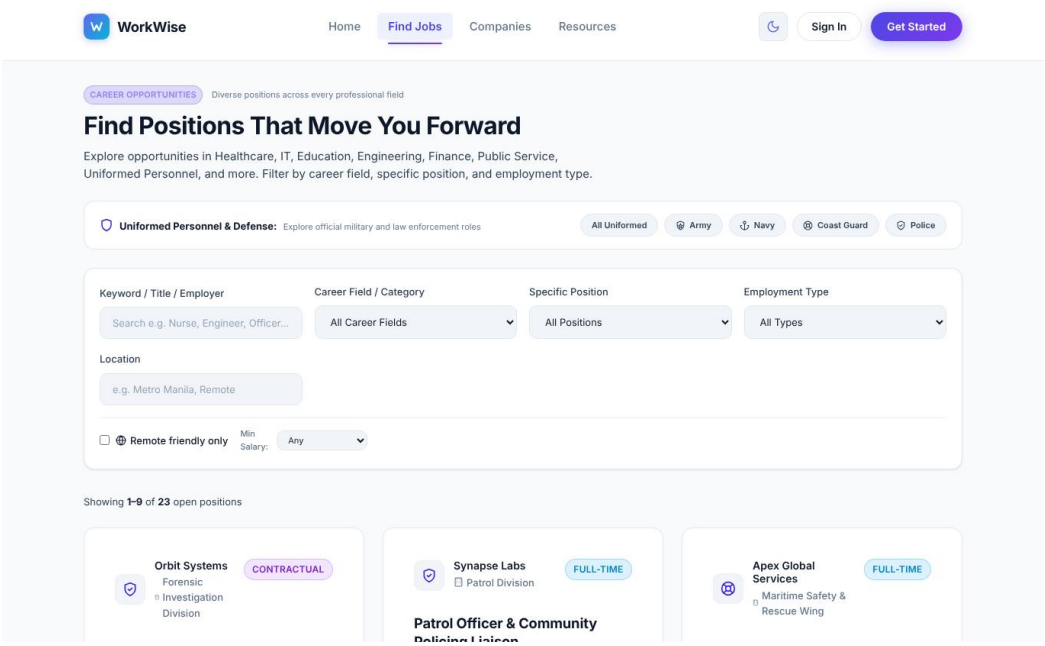


Figure 8. Job Board with Filters

Steps:

1. Open Find Jobs.
2. Choose a category (and branch for uniformed personnel) or type in the search box.
3. Filter by location and salary; results update through HTMX.
4. Select a job card to open its detail page.

7.4 Job Detail

Shows the description, requirements, responsibilities, benefits, salary and company information.

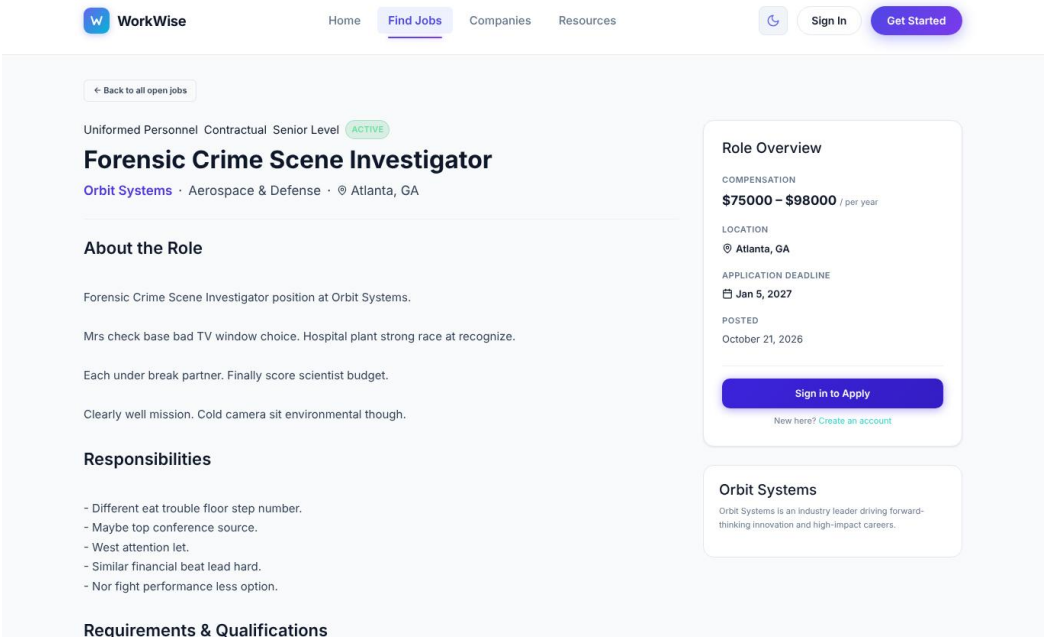


Figure 9. Job Detail Page

Steps:

1. Read the job information.
2. Click Apply (applicants only; visitors are asked to sign in).
3. Use the report option to flag inappropriate content.

7.5 Apply for a Job

The application form collects a cover letter and a resume.

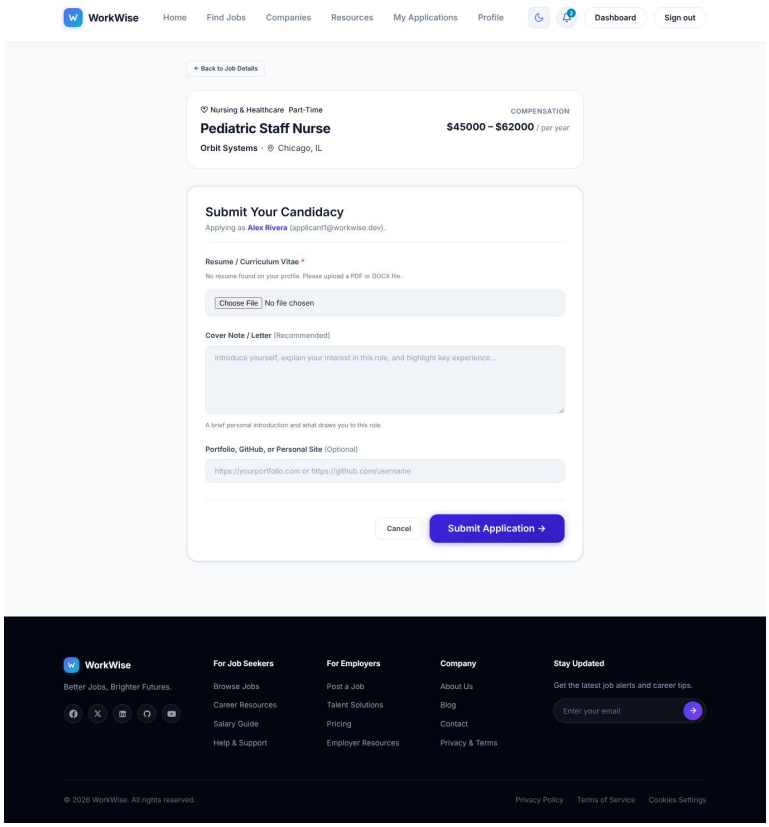


Figure 10. Application Form

Steps:

1. Write your cover letter.
2. Upload a resume (PDF, DOC or DOCX, up to 30 MB) or reuse your profile master resume.
3. Optionally add a portfolio link and submit. A confirmation message and the application detail page are shown.
4. If you have already applied, an informative page is shown instead of the form.

7.6 My Applications

A visual timeline of every application with a status chip.

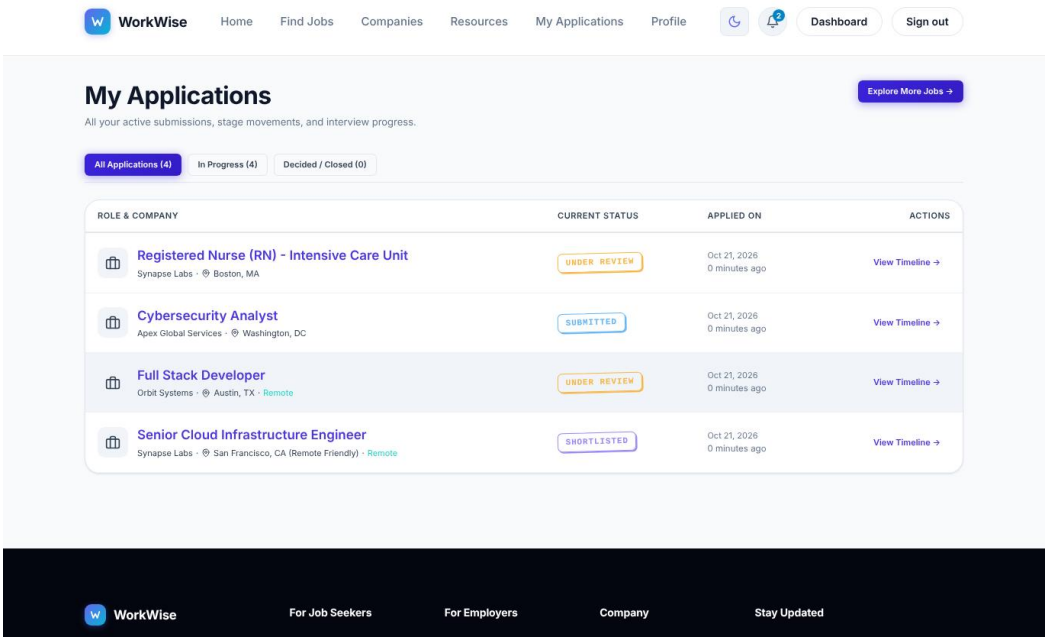


Figure 11. My Applications Page

Steps:

1. Open My Applications from the navigation bar.
2. Review each status (Submitted, Under Review, Shortlisted, Interview Scheduled, Offer Extended, Hired, Rejected).
3. Open an application to see its details or withdraw it.

7.7 Applicant Dashboard and Profile

The dashboard summarises applications, shows upcoming interview tickets and a profile completeness meter.

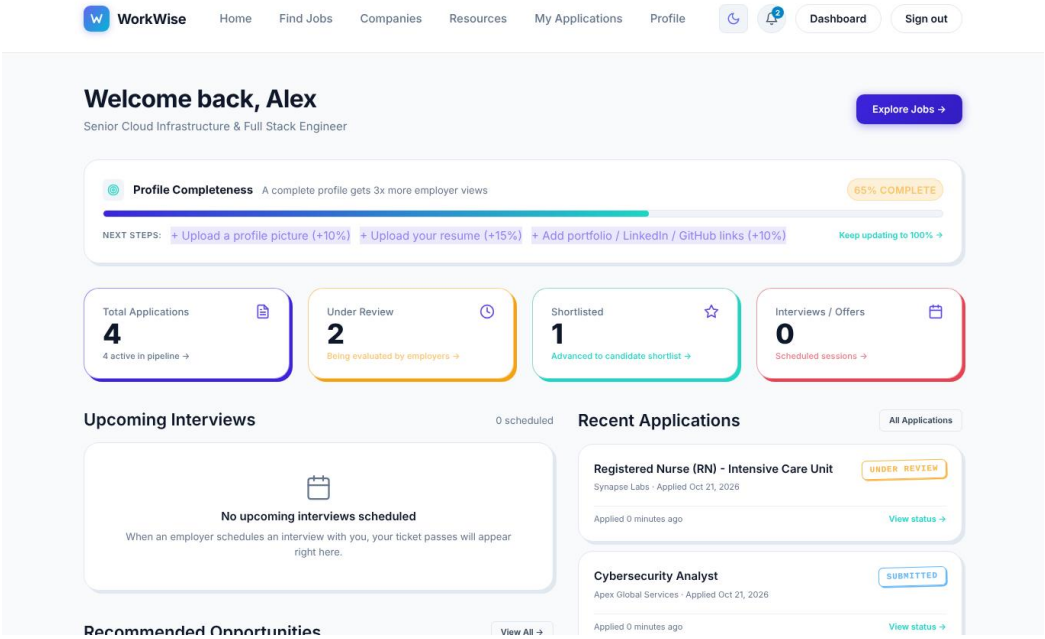


Figure 12. Applicant Dashboard

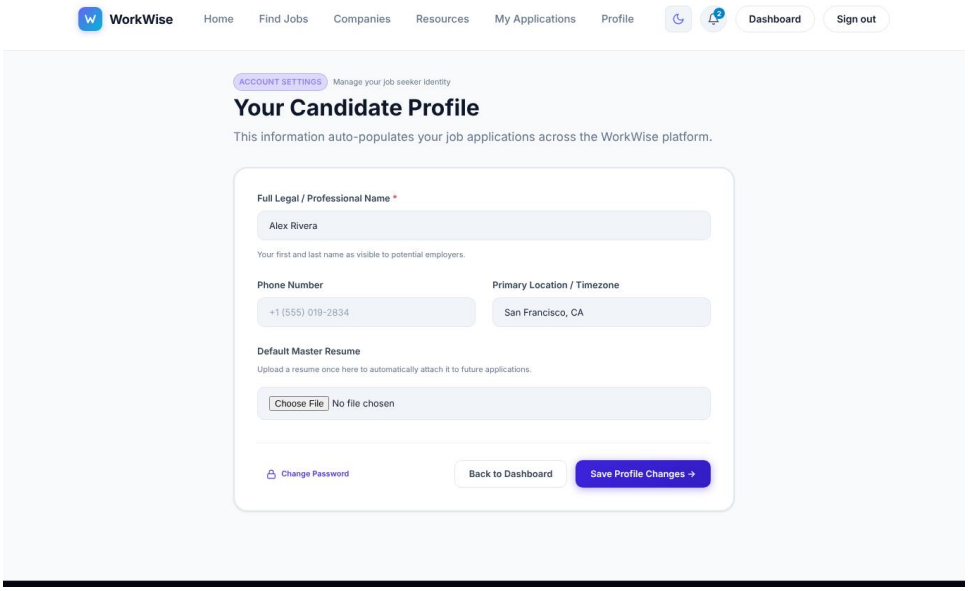


Figure 13. Profile Page

Steps:

1. Sign in as an applicant to land on the dashboard.
2. Follow the Next steps links to upload a photo, resume and portfolio links.
3. Open Profile to edit your headline, skills and default resume.

7.8 Employer Dashboard

Shows open jobs, total applicants, shortlisted candidates and interviews in the next 7 days.

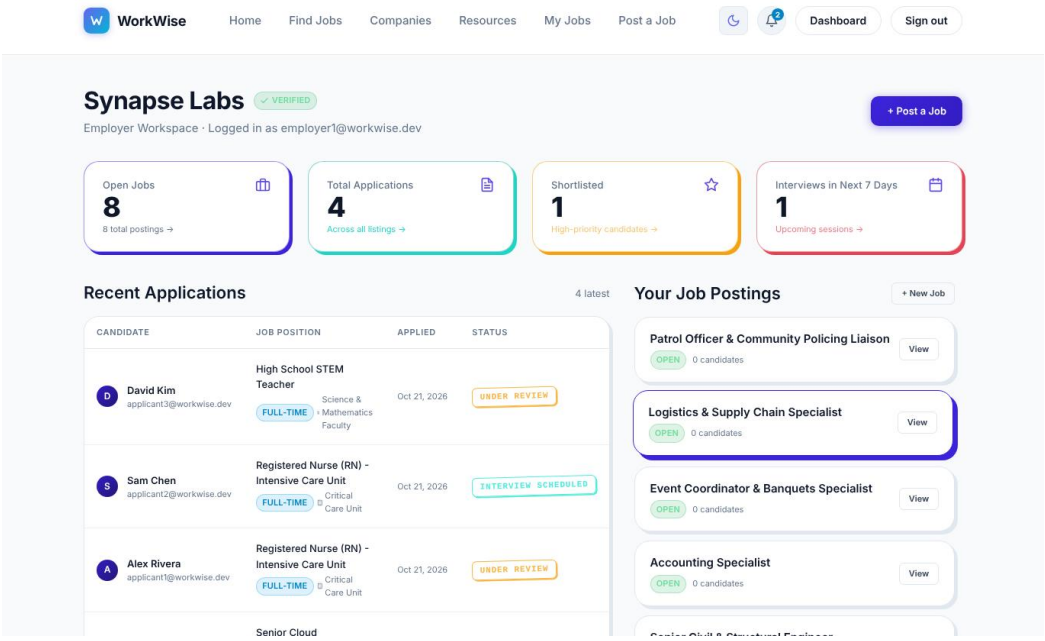


Figure 14. Employer Dashboard

Steps:

1. Sign in as an employer.
2. Review the metric cards and the recent applications list.
3. Use Post a Job or My Jobs to continue.

7.9 Manage Jobs and Post a Job

Employers manage listings by status tab and create jobs with a live applicant-view preview.

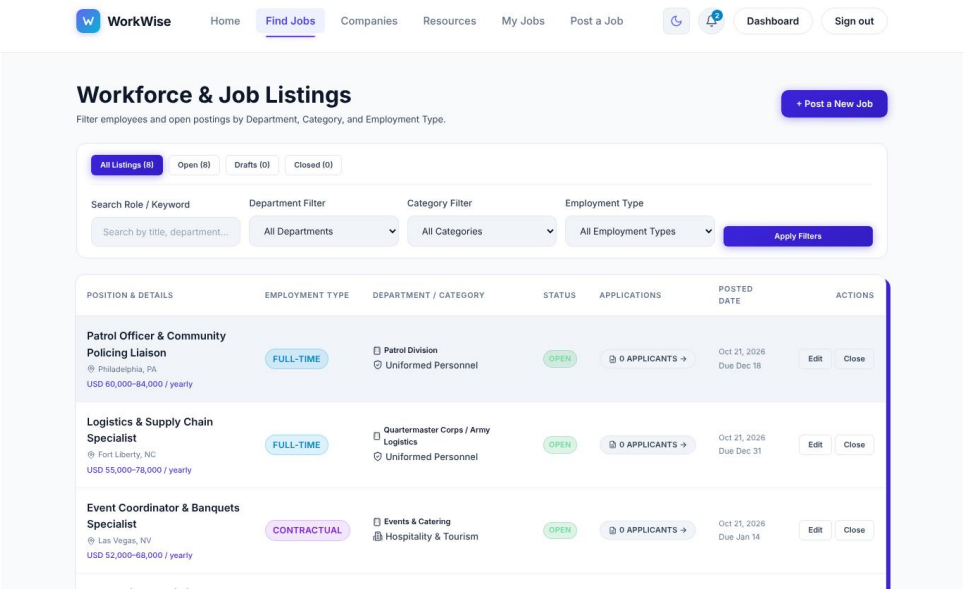


Figure 15. Manage Jobs Page

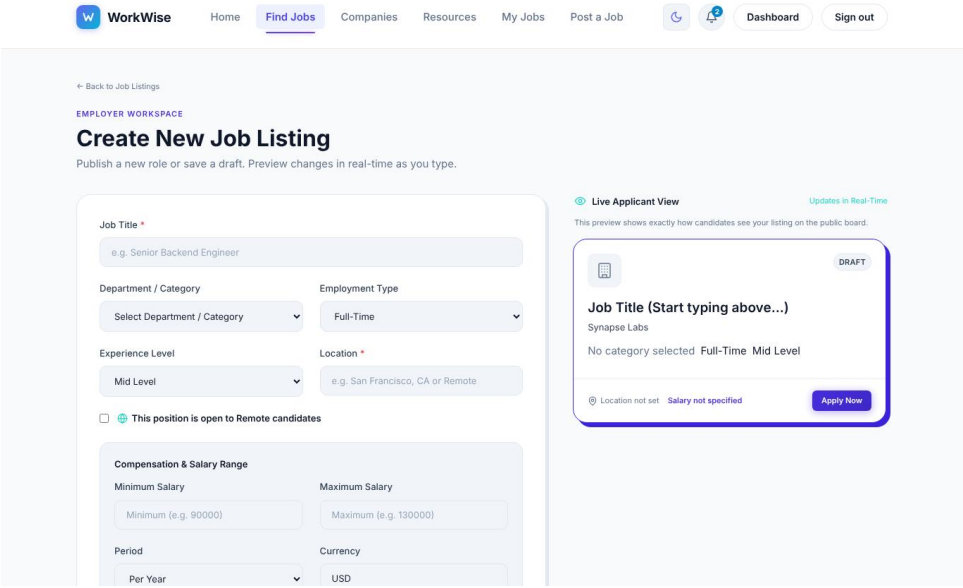


Figure 16. Post a Job Form with Live Preview

Steps:

1. Open My Jobs and use the Draft / Open / Closed / Removed tabs.
2. Select Post a Job, fill in the details and check the preview card.
3. Save as draft or publish (publishing requires a verified employer).
4. Use Edit, Close, Reopen or Delete from the job list.

7.10 Candidate Pipeline

The per-job Kanban board groups candidates by stage.

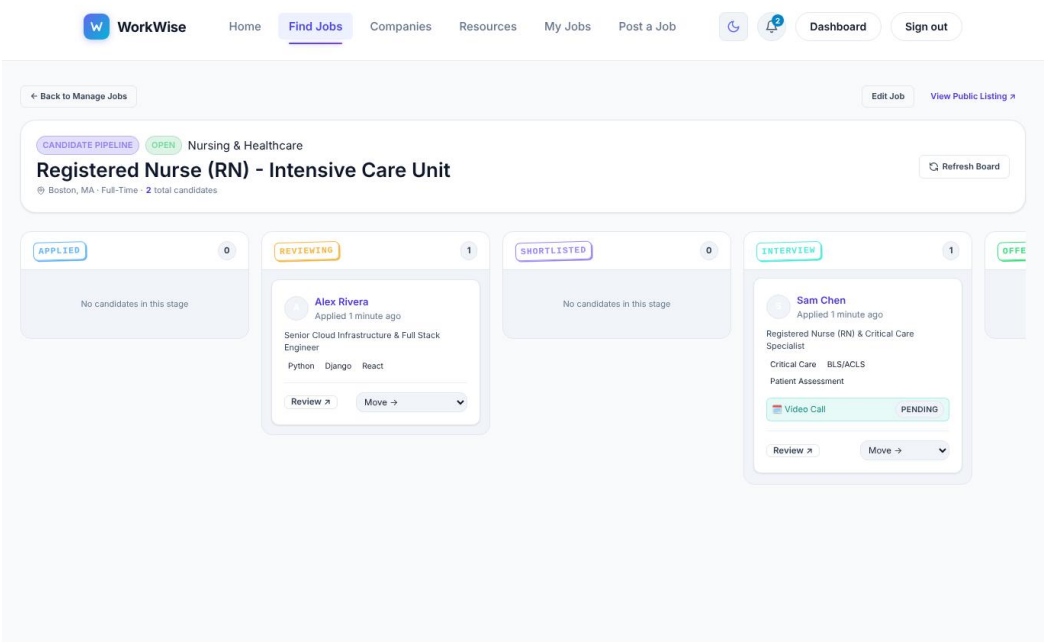


Figure 17. Candidate Pipeline Board

Steps:

1. Open Pipeline from a job in Manage Jobs.
2. Use the Move menu on a card to change stage; only allowed transitions are accepted.
3. Click Review to open the candidate drawer with profile, cover letter and resume download.
4. Schedule an interview and later record its outcome.

7.11 Admin Panel

A standalone control centre for administrators.

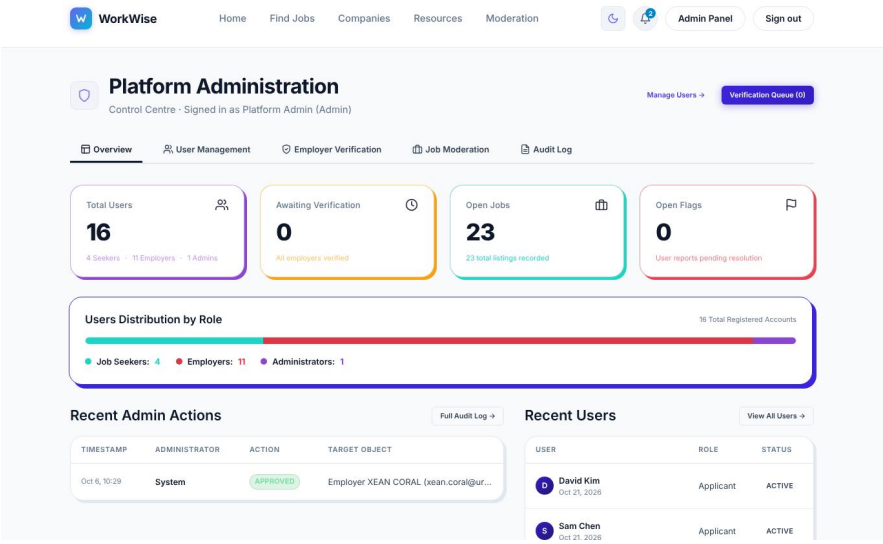


Figure 18. Admin Panel Dashboard

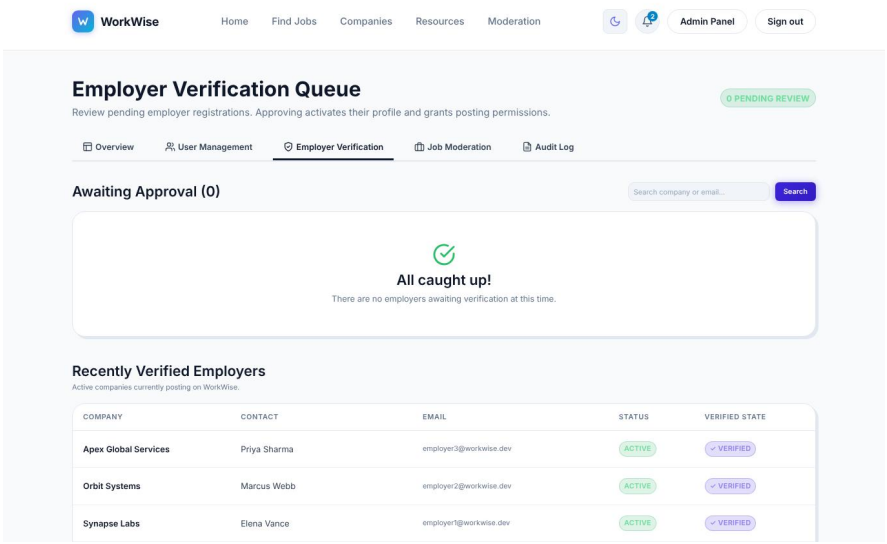


Figure 19. Employer Verification Queue

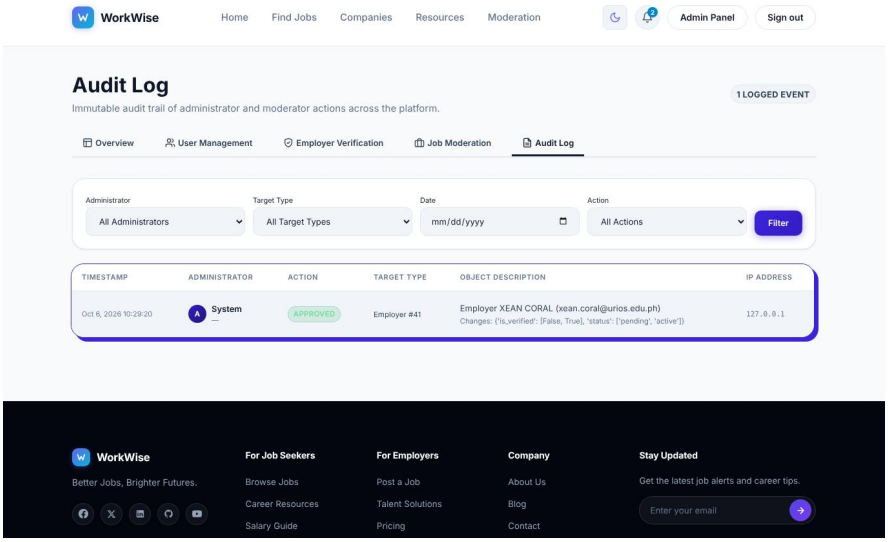


Figure 20. Audit Log

Steps:

1. Sign in as an administrator and open /admin-panel/.
2. Use Verifications to approve or reject employers.
3. Use Users to search, suspend or reactivate accounts, and Jobs to remove postings.
4. Use Audit Log to review administrative actions with filters.

8. Testing

8.1 Testing Approach

Testing combined automated and manual methods:

- **Automated functional and integration tests.** Django's test runner executes 128 tests: accounts (29), jobs (32), applications (24), moderation (19) and core (24). They cover models, constraints, forms, views, role access, the pipeline state machine, moderation, dashboards, notifications and security hardening.
- **Role-based access testing** verifies that each role can reach only its own pages and that ownership rules are enforced.
- **Manual walkthrough** of the seeded demo accounts (applicant, employer, admin) was used to capture the screenshots in Section 7 and confirm that the pages render correctly.

Result: running `python manage.py test` on the repository completed with "Ran 128 tests … OK" and no failures.

8.2 Test Cases and Results

Test ID	Test Scenario	Steps / Input	Expected Result	Actual Result	Status
TC-01	Login with valid credentials	Submit a seeded applicant's email and password	Redirect to the applicant dashboard	Redirected to dashboard	Pass
TC-02	Login rate limiting	Submit wrong passwords repeatedly	HTTP 429 after 5 attempts	HTTP 429 returned (test_login_rate_limiting)	Pass
TC-03	Role profile auto-creation	Save a new employer/applicant user	Matching profile created; none for admin	Profile created as expected (UserSignalTest)	Pass
TC-04	Password reset with wrong code	Submit an invalid verification code	Verification fails	Code rejected (test_verify_invalid_code_fails)	Pass
TC-05	Salary validation	Salary min greater than max	Validation error	Error raised in model and form	Pass
TC-06	Unverified employer publishes job	Employer with pending status posts an Open job	Not allowed; draft only	Publishing refused, draft saved	Pass
TC-07	Intruder edits another job	Different employer requests job edit/close/delete	Access denied	Denied (JobOwnershipTest)	Pass
TC-08	Apply to open job	Applicant submits cover letter and resume	Application created as Submitted	Created (test_apply_to_open_job_successfully)	Pass
TC-09	Duplicate application	Applicant applies to same job twice	Second application blocked	Blocked (test_no_double_apply)	Pass

Test ID	Test Scenario	Steps / Input	Expected Result	Actual Result	Status
TC-10	Apply to closed/removed/expired job	Submit application to unavailable job	Application rejected	Rejected	Pass
TC-11	Valid pipeline transitions	Employer moves card along allowed stages	Status updated and email sent	Updated, email sent	Pass
TC-12	Invalid pipeline transition	Move Hired or skip stages not in ALLOWED_TRANSITIONS	Validation error	Transition rejected	Pass
TC-13	Non-owner moves a card	Other employer posts to transition endpoint	Permission denied	403 returned	Pass
TC-14	Resume validation	Upload .exe or file over 30 MB	Validation error	Rejected; pdf/doc/docx accepted	Pass
TC-15	Protected media	Unrelated user requests a resume URL	403 Forbidden	403 for others; allowed for owner, employer, admin	Pass
TC-16	Moderator suspends user	Moderator tries to suspend a user	Not allowed; superadmin only	Denied (superadmin allowed)	Pass
TC-17	Employer verification	Admin approves/rejects employer	Status changes and audit entry written	Updated and logged	Pass
TC-18	Non-admin opens admin panel	Applicant requests /admin-panel/	Access denied	Denied	Pass
TC-19	Dashboard numbers	Compare dashboard counts with test data	Counts match database	Counts match	Pass
TC-20	Settings split	Load dev and prod settings	Prod has secure cookies, DEBUG off	Verified (test_settings_split_prod_and_dev)	Pass

8.3 Known Issues and Fixes

Issues found during documentation and testing:

1. **Credentials stored in the repository (open).** The .env file and the EmailJS service, template and key identifiers in accounts/email_service.py are committed to GitHub. Fix: move them to environment variables, add .env to .gitignore and regenerate the keys.
2. **README clone URL (open).** The README's quickstart still shows the placeholder URL https://github.com/your-username/inkboard.git. The correct repository is the one listed in Section 9.3.
3. **Stage naming differs between README and code (clarified).** The README names the Kanban stages Applied, Reviewing, Shortlisted, Interview, Offer, Hired and Rejected, while the database statuses are Submitted, Under Review, Shortlisted,

Interview Scheduled, Offer Extended, Hired, Rejected and Withdrawn. The board labels map to these statuses.

4. **Candidate drawer needs internet (open).** HTMX is loaded from the unpkg CDN. When the CDN was unreachable during documentation, the drawer did not open. Fix: serve HTMX from the project's static folder.
5. **Database configuration (open).** The .env file lists a DATABASE_URL, but the settings use SQLite directly; PostgreSQL would require a driver and settings changes.
6. **Duplicate job-type choices (minor).** Job types include both Contract and Contractual with the same label.
7. **Fixed during development:** route ordering in jobs/urls.py (specific routes such as manage/ and post/ are placed before the <slug> routes) to prevent slug collisions; resume validation was centralised in core/validators.py.

9. Deployment and Installation Guide

9.1 System Requirements

- Python 3.11 or newer (the Docker image uses Python 3.12) with pip and venv.
- Git, and a modern web browser (Chrome, Edge or Firefox).
- Internet access for the HTMX script, Google Fonts and EmailJS (the app still runs without it, with reduced interactivity).
- Optional for production: Docker, and a reverse proxy providing HTTPS.
- Minimum hardware: 2 GB RAM, 1 GB free disk space.

9.2 Installation Steps

Step 1: Clone the repository

```
git clone https://github.com/xeancoral31-create/jobapplication2026.git
cd jobapplication2026
```

Step 2: Create a virtual environment and install dependencies

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Step 3: Set up the database (SQLite, no SQL import needed)

```
python manage.py migrate
python manage.py seed_demo --flush     # optional demo data
```

Step 4: Configure environment variables (.env) — do not commit real keys

```
SECRET_KEY=<your-own-long-random-value>
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1
```

Step 5: Run the system and open it in the browser

```
python manage.py runserver
# open http://localhost:8000/
```

Seed accounts (after seed_demo)

Role	Username / Email	Password
Admin	admin (admin@workwise.dev)	admin1234
Employer	employer1@workwise.dev (Synapse Labs)	employer1234
Employer	employer2@workwise.dev (Orbit Systems)	employer1234
Applicant	applicant1@workwise.dev (Alex Rivera)	applicant1234
Applicant	applicant2@workwise.dev (Sam Chen)	applicant1234

Running the tests

```
python manage.py test                # all 128 tests
python manage.py test accounts jobs applications moderation core
```

Docker / production

```
docker build -t workwise:latest .
docker run -d -p 8000:8000 --env-file .env.production workwise:latest
```

The image installs dependencies, collects static files for WhiteNoise, runs as a non-root user and starts Gunicorn with three workers (inkboard.wsgi:application). Set DJANGO_ENV=production (already set in the Dockerfile) so the production settings are used.

9.3 Source Code Repository

GitHub repository: https://github.com/xeancoral31-create/jobapplication2026

The repository contains five Django apps (accounts, jobs, applications, moderation, core), the project package (inkboard), templates, static assets, the Dockerfile and requirements.txt.

10. Team Roles and Contributions

The roles below are a proposed split by module based on the structure of the system. Members must review and edit each row so that it reflects what they actually built, wrote or tested before submission.

Member Name	Role	Specific Contributions
Coral, Xean L.	Project Lead / Backend Developer	Repository owner; project setup and GitHub management; applications and pipeline logic; deployment configuration.
Paredo, Celso	Backend Developer / Database Designer	Models and data dictionary; accounts and authentication; job management module.
Acilo, Angel Mariel M.	Frontend Developer	Templates and Vanilla CSS; dashboards; Kanban board and candidate drawer interface.
Bugasia, Mil Kieven B.	Tester / Security	Automated and manual testing; test cases; validation and access-control checks.
Ramirez, Marrietta P.	Documentation / API Integration	Final project documentation; EmailJS email integration; references and user manual.

11. Conclusion and Recommendations

11.1 Conclusion

Each objective in Section 1.3 was addressed as follows:

1. **Role-based authentication:** met. Applicant, Employer and Admin roles, email-or-username login, suspension enforcement and 5-attempts-per-5-minutes rate limiting are implemented and tested.
2. **Job management:** met. Employers manage jobs through Draft, Open, Paused, Closed, Filled and Removed states with salary and deadline validation and an HTMX live preview.
3. **Discovery and application flow:** met. Applicants filter jobs, submit a cover letter and a validated resume, and cannot apply twice to the same job.
4. **Kanban pipeline:** met. A single service layer enforces ownership and ALLOWED_TRANSITIONS and notifies applicants; interviews and outcomes can be recorded.
5. **Email verification integration:** met. EmailJS REST delivery with a Django email fallback supports the password-reset code flow.
6. **Admin panel:** met. Verification queue, user suspension and reactivation, job moderation and an audit log are available.
7. **Verification and deployment:** met. 128 automated tests pass, and the project includes production settings, WhiteNoise, Gunicorn and a Dockerfile.

Overall, WorkWise demonstrates a complete systems-integration project: multiple Django apps, external services, a controlled data model and a documented deployment path.

11.2 Recommendations

- Move all secrets (including EmailJS keys) to environment variables, remove .env from version control and rotate the exposed keys.
- Serve HTMX and fonts locally to remove the dependency on external CDNs.
- Add PostgreSQL settings and a driver for production, and use DATABASE_URL consistently.
- Provide a documented REST API (for example with Django REST Framework and OpenAPI) so mobile apps or partners can integrate.
- Add real-time notifications (WebSockets or server-sent events), calendar invites for interviews and resume parsing.
- Add continuous integration to run the test suite on every push, and Selenium or Playwright tests for the HTMX interactions.
- Cache and rate-limit with a shared store such as Redis so limits work across multiple Gunicorn workers.

References

Benoit Chesneau. (n.d.). *Gunicorn documentation*. https://docs.gunicorn.org/

Big Sky Software. (n.d.). *htmx documentation*. https://htmx.org/docs/

Clark, A., & contributors. (n.d.). *Pillow documentation*. https://pillow.readthedocs.io/

Docker, Inc. (n.d.). *Docker documentation*. https://docs.docker.com/

Django Software Foundation. (n.d.). *Django documentation*. https://docs.djangoproject.com/

Django Software Foundation. (n.d.). *Django security overview*. https://docs.djangoproject.com/en/stable/topics/security/

EmailJS. (n.d.). *REST API: Send email*. https://www.emailjs.com/docs/rest-api/send/

Evans, D. (n.d.). *WhiteNoise documentation*. https://whitenoise.readthedocs.io/

Mozilla. (n.d.). *HTTP Strict Transport Security (HSTS)*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security

Python Software Foundation. (n.d.). *Python documentation*. https://docs.python.org/3/

SQLite Consortium. (n.d.). *SQLite documentation*. https://www.sqlite.org/docs.html

The Bootstrap Authors. (n.d.). *Bootstrap 5 documentation*. https://getbootstrap.com/docs/5.3/

Appendices

Appendix A – OpenAPI/Swagger specification or Postman collection

Not applicable. WorkWise does not expose a REST API; its HTTP routes are listed in Section 5.2.

Appendix B – Additional diagrams: pipeline state machine

The following transitions are enforced by ALLOWED_TRANSITIONS in applications/services.py.

From status	Allowed next statuses
Submitted	Under Review, Shortlisted, Rejected
Under Review	Shortlisted, Interview Scheduled, Rejected
Shortlisted	Interview Scheduled, Offer Extended, Rejected, Under Review
Interview Scheduled	Offer Extended, Shortlisted, Rejected
Offer Extended	Hired, Rejected
Hired	None (terminal)
Rejected	Under Review (reopen)
Withdrawn	None (terminal)

Appendix C – Sample source code of key modules

Resume validation (core/validators.py, abridged)

```
MAX_RESUME_SIZE_BYTES = 30 * 1024 * 1024  # 30 MB
ALLOWED_RESUME_EXTENSIONS = {".pdf", ".doc", ".docx"}

def validate_resume_file(file):
    if not file:
        return
    file_size = getattr(file, "size", None)
    if file_size and file_size > MAX_RESUME_SIZE_BYTES:
        raise ValidationError("Resume file size must not exceed 30 MB.")
    ext = os.path.splitext(getattr(file, "name", ""))[1].lower()
    if ext not in ALLOWED_RESUME_EXTENSIONS:
        raise ValidationError("Only PDF, DOC and DOCX files are accepted.")
```

Pipeline transition check (applications/services.py, abridged)

```
def transition_application_status(application, new_status, user):
    if not user.is_authenticated or (application.job.employer != user
                                     and not user.is_staff):
        raise PermissionDenied("Only the job's employer can move cards.")
    allowed = ALLOWED_TRANSITIONS.get(application.status, [])
    if new_status not in allowed:
        raise ValidationError("Invalid transition ...")
    # update status, notify applicant, send email
```

Appendix D – User acceptance or survey forms

Not applicable. No survey or user acceptance forms were used for this documentation.

Appendix E – AI-use disclosure

The AI assistant Claude (Anthropic) was used to help draft and format this documentation from the project's GitHub repository, README and the course template, to generate the architecture, data-flow and ERD diagrams, and to capture screenshots of the running

application with seeded demo data. The test results come from running the repository's own test suite. The project members are responsible for reviewing the content, in particular Section 10 (Team Roles and Contributions).